用户认证与权限控制

Authentication and Authorization

课程名称: Java Web 应用开发 | 第十五讲
授课方式: 双语教学 (Bilingual Teaching)
课程性质: 专业核心课程 | 2学分/32学时
学期: 2025年秋季

CONTENT

本讲概要

Course Outline

1 数据驱动导入

2 安全攻防闯关

3 AI代码评审对战

4 核心概念精讲

5 修复与验证

6 课堂总结

教学目标

Learning Objectives

知识目标 (Knowledge Objectives)

- 区分认证和授权的概念与流程
  (Distinguish authentication and authorization)
- 掌握基于会话的登录态维护机制
  (Master session-based login state maintenance)
- 掌握RBAC基于角色的访问控制思路
  (Master RBAC role-based access control)

能力目标 (Ability Objectives)

- 能够实现登录、登出和会话管理
  (Be able to implement login, logout and session)
- 能够编写基于角色的权限过滤器
  (Be able to write role-based filter)
- AI辅助 能使用AI工具进行安全漏洞识别与代码审查，并批判性评估AI建议
- AI辅助 能在AI攻防闯关中独立发现和修复常见Web安全漏洞

AI教学工具矩阵

AI Teaching Tool Matrix

本讲AI工具全景 (AI Tool Overview for This Lesson):

工具 Tool	阶段 Stage	作用 Role
WebDev助教 (智能体)	课前+课中	学情分析、实时问答、个性化支持
WebSec挑战 (攻防闯关)	课中	AI生成漏洞代码，安全闯关训练
TeamCoach (项目教练)	课前+课后	团队贡献追踪、薄弱点预警
Code Review Battle (评审对战)	课中	人机协同代码审查、批判性思维

本讲覆盖全部6个AI教学情境：①学情数据采集与分析 ②数字资源整合与运用 ③适配的教学场景设计 ④多维智能评价反馈 ⑤师生机协同教学 ⑥个性化学习支持

Part I: Data-Driven Introduction

第一部分：数据驱动导入

WebDev助教 · 课前学情数据

AI辅助环节 · WebDev助教

课前学情数据展示 (Pre-class Learning Analytics):

智能体后台数据 · 课前3天采集

- 全班58人中 **42人** 向WebDev助教提问，覆盖率72%
- 热点问题 TOP 3：
  i. Session过期后如何处理？（23人）
  ii. Filter如何判断用户角色？（18人）
  iii. Cookie和Session的安全区别？（15人）
- 薄弱点：多数学生混淆 `invalidate()` 和 `removeAttribute()`

教学决策: 据此调整今天教学重点——强化Session安全管理和角色权限控制

TeamCoach周报 + 权限漏洞演示

AI辅助环节 · TeamCoach

TeamCoach上周报告摘要 (Weekly Report Highlights):

- 第3组：Service层无人负责，可能成为集成瓶颈
- 第5组：赵六连续两周仅修改README，贡献不足
- 全班测试覆盖率为0%——今天的攻防闯关会让你体会测试的价值

现场演示 (Live Demo) — 课程作业管理系统:

测试1: 用 `admin / admin123`（管理员）登录 → 正常进入管理后台 ✓

测试2: 用 `tom / tom123`（普通用户）登录 → **也能进入管理后台** ✗

> **核心问题：能登录 ≠ 能安全控制访问**
>
> *Is this system secure if a regular user can access the admin panel?*

Part II: WebSec Challenge

第二部分：安全攻防闯关

WebSec挑战 · 平台介绍

WebSec Challenge · Platform Overview

AI辅助环节 · WebSec挑战

5个安全关卡 (5 Security Levels):

关卡	漏洞类型	知识点	安排
第1关	SQL注入	PreparedStatement vs 拼接	✅ 课前完成
第2关	**越权访问**	**Filter权限检查缺失**	**课堂重点**
第3关	Session劫持	Session固定攻击	✅ 课前完成
第4关	XSS跨站脚本	输出转义、`<c:out>`	课后
第5关(BOSS)	综合漏洞	全部 + CSRF	课后

规则: AI为每人生成不同漏洞代码 → 找漏洞 → 写修复 → 自动测试 → 过关

课前已通过平台完成第1、3关；今天聚焦第2关——与本讲"权限控制"直接相关

第2关 · 越权访问漏洞

Level 2 · Broken Access Control

漏洞代码 (Vulnerable Code) — OWASP A01:2021:

```java
@WebServlet("/admin/users")
public class AdminUserServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException, ServletException {
        HttpSession session = req.getSession(false);
        if (session != null && session.getAttribute("user") != null) {
            // 只检查了"已登录"，没有检查角色!
            List<User> users = userService.findAll();
            req.setAttribute("users", users);
            req.getRequestDispatcher("/admin/userList.jsp").forward(req, resp);
        } else {
            resp.sendRedirect(req.getContextPath() + "/login.jsp");
        }
    }
}
```

Part III: Code Review Battle

第三部分：AI代码评审对战

Code Review Battle · 规则

Code Review Battle · Rules

AI辅助环节 · Code Review Battle

学生 vs AI · 谁能发现更多安全漏洞？

对战流程 (Battle Flow):

第1轮 · 人工审查（5min）：各组分析代码，找出安全问题
第2轮 · AI审查（2min）：教师将代码提交给AI，投影展示
第3轮 · 人 vs AI 对比辩论（8min）：对比发现，讨论采纳

评审对象: `LogoutServlet.java` — 已知存在 **4个安全漏洞**
你能找到几个？AI又能找到几个？

待审查代码 · LogoutServlet.java

Code Under Review · LogoutServlet.java

```java
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.removeAttribute("user");
        }
        resp.sendRedirect(req.getContextPath() + "/login.jsp");
    }
}
```

任务: 审查这段代码，找出其中的安全隐患，记录问题位置、风险描述、修复方案

分组任务

Group Assignment

AI辅助环节 · Code Review Battle

三组并行审查 (Three Groups, Parallel Review):

A组：认证组	B组：授权组	C组：视图一致性组
审查 `LoginServlet` `LogoutServlet`	审查 `RoleBasedAuthFilter`	审查 `index.jsp` 菜单
Session管理是否安全？	权限过滤是否按角色控制？	菜单显示与权限一致？

审查输出格式 (Report Format):

1. 问题位置（行号/方法名）
2. 风险说明（可能造成什么后果）
3. 对应知识点（Authentication / Authorization / RBAC / Session / Filter）
4. 修复建议

Part IV: Core Concepts

第四部分：核心概念精讲

认证与授权

认证 (Authentication)

问题: 你是谁？

Who are you?

过程:

- 用户提供凭证（用户名+密码）
- 系统验证身份
- 确认用户身份

比喻: 身份证检查

ID verification

授权 (Authorization)

问题: 你能做什么？

What can you do?

过程:

- 检查用户角色/权限
- 判断是否有访问权限
- 允许或拒绝访问

比喻: 门禁卡权限

Access card permission

Session认证完整流程

请求流程 (Request Flow):

1. 用户提交登录表单
   User submits login form
2. **LoginServlet** 验证用户名密码
   Verify credentials
3. 验证通过 → 创建Session，存储User对象
   Create session, store user object
4. 后续每次请求 → **Filter** 拦截检查角色
   Filter intercepts, checks role
5. 角色匹配 → 放行；不匹配 → 403
   Match → allow; Mismatch → deny

组件职责 (Component Responsibilities):

组件	职责	阶段
LoginServlet	验证身份	认证
Session	维持登录状态	状态管理
Filter	拦截检查权限	授权
JSP	按角色渲染界面	展示

核心原则: 每个组件只做一件事

Single Responsibility Principle

LoginServlet · 登录实现

登录核心逻辑 (Login Core Logic):

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws IOException, ServletException {
        String username = req.getParameter("username");
        String password = req.getParameter("password");
        User user = userService.authenticate(username, password);
        if (user != null) {
            HttpSession session = req.getSession();
            session.setAttribute("user", user);   // 只存一个对象
            resp.sendRedirect(req.getContextPath() + "/index.jsp");
        } else {
            req.setAttribute("error", "用户名或密码错误");
            req.getRequestDispatcher("/login.jsp").forward(req, resp);
        }
    }
}
```

登出与Session安全

安全登出 (Secure Logout):

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.invalidate();  // 彻底销毁Session
        }
        resp.sendRedirect(req.getContextPath() + "/login.jsp");
        }}
```

关键区别 (Key Difference):

- `removeAttribute("user")` — 只删一个属性，Session仍存活
- `invalidate()` — **彻底销毁**整个Session，所有属性一并清除

JSP按角色显示菜单

Role-Based Menu Display

动态菜单 (Dynamic Menu):

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:if test="${sessionScope.user != null}">
    <p>欢迎, ${sessionScope.user.displayName}</p>
    <a href="${pageContext.request.contextPath}/logout">退出</a>
</c:if>
<c:if test="${sessionScope.user.role == 'ADMIN'}">
    <a href="${pageContext.request.contextPath}/admin/panel.jsp">管理后台</a>
</c:if>
```

核心原则 (Core Principle):

> 前端隐藏菜单 = 用户体验优化（UX） 后端Filter拦截 = **真正的安全边界**
>
> **完整方案 = 前端隐藏 + 后端控制**

Part V: Fix & Verify

第五部分：修复与验证

权限过滤器修复后代码

基于角色的Filter *Role-based filter*

```java
@WebFilter("/admin/*")
public class RoleBasedAuthFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);
        if (session == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }
        User user = (User) session.getAttribute("user");
        if (user != null && "ADMIN".equals(user.getRole())) {
            chain.doFilter(req, res);   // ADMIN放行
        } else if (user != null) {
            response.sendError(403);    // 非ADMIN → 403
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

最小测试清单

Verification Checklist

编号	测试场景	操作	预期结果
T1	未登录访问后台	直接访问 `/admin/panel`	重定向到登录页
T2	普通用户越权	`tom/USER` 登录后访问 `/admin/panel`	返回 **403**
T3	管理员正常访问	`admin/ADMIN` 登录后访问 `/admin/panel`	正常进入
T4	退出后再访问	退出后再访问 `/admin/panel`	重定向到登录页
T5	菜单一致性	不同角色查看首页菜单	仅ADMIN显示"管理后台"

现场验证 (Live Verification):

- 逐项执行T1-T5，现场演示修复效果

修复前后对比

Before vs After

组件	问题版 (Before)	修复版 (After)
RoleBasedAuthFilter	只检查`isLoggedIn`	检查`user.getRole()=="ADMIN"`
index.jsp	登录即显示管理入口	仅ADMIN显示
LogoutServlet	`removeAttribute`	`invalidate()`
LoginServlet	冗余存储4个属性	只存`user`对象

知识点映射 (Knowledge Mapping):

- 修复1 → **Authorization + RBAC + Filter**
- 修复2 → **View Rendering + Session**
- 修复3 → **Session Lifecycle + Logout**
- 修复4 → **Session Design + Data Consistency**

课程思政：制度与秩序

Ideological Education: System & Order

RBAC与社会治理 (RBAC & Social Governance)

技术层面 (Technical):

- 认证：你是谁？ *Who are you?*
- 授权：你能做什么？ *What can you do?*
- 原则：最小权限、默认拒绝

社会治理层面 (Social Governance):

> **制度先行，权责明晰 (System First, Clear Responsibilities)**
> 任何系统都需要规则和边界
> 用户分类、权限隔离，保障公平与安全

> **依法治网 (Governing Cyberspace by Law)**:
> 技术人员的责任：保护用户信息，遵守法律法规
> 维护网络空间秩序，保护公民权益

AI伦理反思

AI Ethics Reflection

从Code Review Battle看AI的能力边界

AI能做到的 (AI Can):

- 快速扫描，覆盖面广
- 模式匹配能力强，不遗漏常见漏洞
- 标准化输出，格式清晰

AI做不到的 (AI Cannot):

- 理解业务上下文和系统设计意图
- 判断"合理的设计折衷" vs "真正的安全漏洞"
- 承担代码安全的最终责任

> **AI是助手，不是裁判 (AI Assists, Not Replaces Judgment)**
> 技术决策的最终责任在人——这既是技术原则，也是职业伦理

Part VI: Summary

第六部分：课堂总结

本节课数据回顾

Lesson Data Review

AI辅助环节 · 全部4个工具

课堂结束 — 打开各平台查看本节课实际数据

攻防闯关 (WebSec Challenge):

- 各关卡最终通过率（干预前 vs 干预后）
- 全班典型错误类型分布
- 闯关排行榜展示

评审对战 (Code Review Battle):

- 各组发现漏洞数 vs AI发现数
- AI是否存在误报？原因是什么？
- 人机协同的净增发现

WebDev助教: 课堂实时提问统计与高频问题回顾

三个核心收获

Three Key Takeaways

今天的三个核心收获 (Three Key Takeaways):

1. 认证回答"你是谁"，授权回答"你能做什么"

- Authentication ≠ Authorization
- 登录成功 ≠ 拥有所有权限

2. RBAC + Filter = 后端权限控制的基本骨架

- Filter是安全的第一道防线
- 前端隐藏 + 后端控制 = 完整方案

3. AI辅助 人机协同 > 单独任何一方

- AI发现了Cache-Control问题，但也产生了1个误报
- WebSec闯关让每个人都亲手修复了漏洞
- 批判性判断 > 盲目采纳

课后任务

课后实践 (After-Class Practice):

序号	任务	相关工具
1	完成 WebSec挑战 第4-5关（XSS + 综合BOSS关）	WebSec挑战
2	团队项目中实现权限模块	TeamCoach将追踪
3	向 WebDev助教 提问今天课堂中的疑惑	WebDev助教
4	**AI使用反思（必做）**	—

AI使用反思要求:

1. AI在攻防闯关中给你的提示有帮助吗？你如何判断是否采纳？
2. Code Review Battle中AI的误报说明了什么？
3. 在你未来的开发工作中，AI代码审查应扮演什么角色？

感谢聆听！(Thank You!)

XXX学院

Java Web 应用开发